

TokenShare Real Plugin Experiment Design

基于最新仓库进展的 factorization 插件实验与真实插件泛化实验

TokenShare Project

2026-06-28

1 本文档更新点

本文档根据当前 GitHub 仓库 **BriasedTu/TokenShare** 的最新实现状态重写实验设计。相比上一版，核心变化有五点：

1. 不再使用简化 toy / stub 实验来支撑论文主张。实验必须围绕真实插件边界、真实 descriptor、真实 artifact、真实 parser / verifier / merge policy 和真实 protocol lifecycle 展开。
2. factorization 插件已经从早期的简单 range demo 更新为 **factorization@0.1.0** 第一切片：包含 schema、pure models、descriptor、candidate range partition、parser、range verifier、all-required merge policy、prime / semiprime fixture flow、prompt package builder 和 AI output parse policy。
3. factorization 的核心 unit type 不再只是简单 range task，而是 **root**、**factor_integer**、**factor_search_range**、**factorization_merge** 四类任务单元。
4. range worker 的输出 contract 使用 **factorization.range_result.v1**，结果种类是 **found_factor** 或 **no_factor_in_range**；所有整数必须以 decimal string 保存，避免 JSON 大整数语义不清。
5. 第四个实验继续测试协议泛化能力，但必须使用真实插件：**factorization plugin + 真实 Lean proof plugin / Lean adapter**。如果 Lean 插件尚未实现，则 Experiment 4 只能标记为待运行，不能用 lean_stub 结果替代。

2 最新仓库事实摘要

当前实验设计以以下仓库事实为约束：

- V1 的目标是本地可复现实验内核：任务注册、递归图、lease、attempt、artifact、插件化验证、canonical output、merge、settlement 和 JSONL event replay。
- 协议核心不理解 factorization、Lean 或 structured report 的领域逻辑；领域规则只属于插件。
- factorization 插件第一版不实现 **one_success**、early success、sibling pruning 或 composite cofactor 的完整递归 resolution。

- Phase 7 实验级 AI API executor 已实现; executor 只负责 transport、raw output、provenance、usage/cost、parser bridge, 不拥有领域解析权。
- Lean 插件新口径是真实 Lean checker 驱动的形式化证明插件, 不是 stub。验证必须使用固定本地 Lean / lake / toolchain / library context, 并持久化 checker 日志和环境摘要。

3 实验总目标

四个实验分别回答四个问题:

实验	问题	成功证据
Experiment 1	factorization 真实插件能否端到端跑通?	prime / semiprime fixture 走完 descriptor freeze、split、execution、verification、canonical、all-required merge、settlement。
Experiment 2	局部失败和错误输出是否会污染 canonical state?	invalid factor、false no-factor、parse failure、crash / expired lease 均被拒绝或恢复。
Experiment 3	关键协议机制是否必要?	关闭 verification、parser policy、requeue、all-required merge gate 后出现预期退化。
Experiment 4	协议核心是否能泛化到不同真实插件?	factorization 与真实 Lean plugin 共享同一套 protocol lifecycle, 且 core 不硬编码任何领域规则。

4 Factorization 插件最新契约

4.1 插件身份与策略 ID

字段	最新值
plugin_id	factorization
plugin_version	0.1.0
split_strategy_id	factorization.candidate_range_partition.v1
validator_policy_id	factorization.range_result.validator.v1
merge_policy_id	factorization.all_required_range_merge.v1
parser_id	factorization.range_result.parser.v1
prompt builder	factorization.build_factor_search_prompt_package.v1

4.2 支持的 TaskUnit 类型

```
root
factor_integer
factor_search_range
factorization_merge
```

说明: `root` 是兼容现有 `TaskUnit.create_root()` 的入口。插件通过 `root input` 和 `TaskSpec.split_s` 将其解释为 `factor_integer` 根目标。真正的 `range` 子任务使用 `factor_search_range`。

4.3 关键 schema version

对象 / Artifact	Schema version
RootInput	factorization.root_input.v1
FactorIntegerSubject	factorization.factor_integer_subject.v1
CandidateRangePartitionParams	factorization.candidate_range_partition_params.v1
CandidateRangeCoverageProof	factorization.candidate_range_coverage_proof.v1
FactorSearchRangeInput	factorization.factor_search_range_input.v1
FactorSearchInstruction	factorization.factor_search_instruction.v1
PromptPackage	phase3.prompt_package.v1
RangeResult	factorization.range_result.v1
FactorizationMergeResult	factorization.merge_result.v1
PrimeFactorizationResult	factorization.prime_factorization_result.v1

5 统一生命周期与事件要求

实验必须使用已有 Phase 3-5 events, 不新增 factorization 专用 event type。权威事实进入以下事件和 artifacts:

```
REGISTRY_SNAPSHOT_RECORDED
EXECUTION_REQUEST_RECORDED
EXECUTION_SUBMISSION_RECORDED
VERIFICATION_RECORDED
CANONICAL_OUTPUTS_BOUND
SPLIT_STRATEGY_INVOCATION_RECORDED
DECOMPOSITION_PROPOSAL_RECORDED
EXPANSION_DECISION_RECORDED
MERGE_PLAN_RECORDED
TASK_EXPANDED
MERGE_TASK_LINK_RECORDED
MERGE_RECORDED
EXPECTED_OUTPUT_RESOLVED
CONTRIBUTION_STATE_CHANGED
SETTLEMENT_RECORDED
```

每个 run 至少保存：

```
outputs/events/<run_id>.jsonl
outputs/artifacts/<run_id>/descriptor.json
outputs/artifacts/<run_id>/registry_snapshot.json
outputs/artifacts/<run_id>/root_input.json
outputs/artifacts/<run_id>/factor_integer_subject.json
outputs/artifacts/<run_id>/range_inputs/*.json
outputs/artifacts/<run_id>/execution_instructions/*.json
outputs/artifacts/<run_id>/prompt_packages/*.json
outputs/artifacts/<run_id>/raw_outputs/*.json
outputs/artifacts/<run_id>/parsed_outputs/*.json
outputs/artifacts/<run_id>/verification_reports/*.json
outputs/artifacts/<run_id>/merge_records/*.json
outputs/artifacts/<run_id>/settlement_records/*.json
outputs/metrics/<experiment>_summary.csv
```

6 Experiment 1: Factorization End-to-End Execution

6.1 实验目的

验证 factorization 真实插件可以在现有协议核心上完成完整闭环。Experiment 1 不注入失败，重点检查：

1. root input 不是最终结果，必须先生成并 canonical 绑定 **factor_integer_subject**。
2. split strategy 只能消费 canonical subject，并产生完整候选域 range partition。
3. 每个 range child 都必须有 **ExecutionInstruction**；AI/mock-AI 路径还必须有插件生成的 **PromptPackage**。
4. range output 必须经过 parser / verifier，再绑定 canonical。
5. merge 只在 all required range slots 都 canonical 后启动。
6. semiprime fixture 只有在 factor 与 cofactor 均通过 primality evidence 时才能解析为 **PrimeFactorization**。

6.2 推荐 fixture

必须优先使用仓库中已有的 Phase 6 fixture，而不是手写 toy 数据。

Fixture	目标数	预期结果
small prime direct complete	2 或 3	split strategy 返回 complete，不生成 proposal、 merge plan、child ranges 或 merge task。

prime range flow	97	range children 全部 no_factor_in_range, merge 输出 prime_certificate 和 PrimeFactorizationResult([97])。
semiprime range flow	$91 = 7 \times 13$	found factor range 和 no-factor ranges 都经 verifier, all-required merge 后输出 PrimeFactorizationResult([7,13])。
extended semiprime benchmark	$8051 = 83 \times 97$	可作为更大可读展示 run, 但不能替代已有 fixture 回归。

6.3 Semiprime run 完整流程

以 target_n=91 为主线:

Step	操作	必须检查
1	冻结 registry / descriptor	registry snapshot 中包含 factorization@0.1.0, descriptor digest 稳定。
2	注册 root task	RootInput 使用 factorization.root_input.v1, requested_output=prime_factorization_result。
3	生成 root canonical subject	deterministic fixture executor 生成 FactorIntegerSubject, 经 verify_factor_integer_subject() 后 canonical 绑定, bundle key 必须是 factor_integer_subject, 不是最终输出名。
4	调用 split strategy	记录 SPLIT_STRATEGY_INVOCATION_RECORDED; strategy 为 factorization.candidate_range_partition.v1。
5	生成 proposal 和 merge plan	DecompositionProposal child 只能是 factor_search_range; coverage proof 必须 no gap、no overlap、full domain covered、sqrt bound checked。
6	创建 range children	每个 child 拥有 FactorSearchRangeInput, 包含 coverage_id、child_index、child_count 和 partition_params_digest。
7	生成 execution request	每个 range child request 同时引用 execution_instruction_ref 和插件生成的 prompt_package_ref。
8	执行 bounded range search	executor 只搜索自己被分配的闭区间, 不得搜索区间外, 不得创建子任务, 不得声称最终 prime factorization。
9	保存 raw / parsed artifacts	AI/mock-AI 路径必须保存 raw output; parser 成功后保存 typed RangeResult artifact。

10	插件 verifier 检查	found_factor 校验 range、divisibility、cofactor、coverage/params; no_factor_in_range 做 deterministic recheck, 并受 budget 约束。
11	canonical binding	只有 verifier passed 的 range_result 可以 canonical。
12	all-required merge gate	只有所有 required range slots canonical 后, 才创建 merge task; partial run 不能 merge。
13	merge policy	merge_required_range_results() 选择数值最小 verified factor; 若 factor/cofactor 均 prime, 生成 PrimeFactorizationResult 。
14	expected output resolution	只有 prime_certificate 或完整 prime_factorization_result 可以解析为 parent output; nontrivial_factor_found 不得伪装成 final。
15	completion / settlement	写 parent completion、contribution state change、settlement, 并保证 event replay 可重建。

6.4 Experiment 1 指标

Metric	Definition
subject_canonical_success	root / factor_integer subject verified and canonical-bound.
range_coverage_valid	coverage proof no gap / no overlap / full domain / sqrt bound all true.
range_verification_rate	verified range results / submitted parsed range results.
prompt_package_coverage	range execution requests with plugin-owned prompt package / range execution requests.
all_required_merge_gate_success	merge starts only after all required range slots canonical.
final_correctness	final prime factor multiset product equals target n.
settlement_success	settlement event exists and references verified / merged contributions.
ledger_replay_success	lifecycle reconstructs from JSONL and artifacts.

7 Experiment 2: Failure Injection and Recovery

7.1 实验目的

验证错误输出、parse failure、crash 和 budget failure 不会污染 canonical state。每个 failure case 必须单独 run, 不能混合注入。

7.2 Case A: invalid found_factor

注入一个结构化但错误的 RangeResult:

```
{
  "schema_version": "factorization.range_result.v1",
  "result_kind": "found_factor",
  "target_n": "91",
  "range_start": "2",
  "range_end": "5",
  "found_factor": "5",
  "cofactor": "18",
  "checked_divisor_count": 4
}
```

预期:

```
VERIFICATION_RECORDED status = rejected
failure_kind = invalid_output
no CANONICAL_OUTPUTS_BOUND for this attempt
unit later requeued or superseded by a valid attempt
```

7.3 Case B: false no_factor_in_range

对包含真实 divisor 的 range 恶意提交 no_factor_in_range。例如 target=91, range=[7,7], 实际 7 整除 91。

预期: verifier deterministic recheck 发现 divisor, 拒绝该 output。该 case 是 factorization verifier 可信度的关键证据。

7.4 Case C: parse failure / raw-only forbidden

AI executor 或 mock-AI executor 返回 free-form raw text:

```
I found factor 7.
```

由于 descriptor 声明:

```
parser_id = factorization.range_result.parser.v1
parse_required = true
raw_only_allowed = false
```

预期:

```
raw output artifact persisted
parse failure artifact persisted
candidate_output_refs = {}
no VERIFICATION_RECORDED for successful range_result
no CANONICAL_OUTPUTS_BOUND
```

7.5 Case D: worker crash / expired lease

让某个 range worker 获得 lease 后不提交 output。预期：lease expired 后该 unit 回到 Ready，并由新 attempt 完成。crashed attempt 没有 submission，因此不能进入 verification / canonical。

7.6 Case E: no-factor recheck budget exceeded

构造一个 range，使 `no_factor_recheck_max_divisors` 小于 range 长度。预期 verifier rejected，不允许无界 brute-force loop，也不允许把无法 recheck 的 no-factor claim canonical。

7.7 Experiment 2 指标

Metric	Definition
detection_rate	rejected injected invalid outputs / injected invalid outputs.
parse_failure_isolation_rate	parse failures not entering verification/canonical / parse failure cases.
false_accept_rate	invalid or raw-only outputs becoming canonical / injected invalid outputs.
requeue_success_rate	expired/rejected required units later re-leased and verified / expired/rejected required units.
canonical_pollution_count	invalid/crashed/raw-only attempts bound as canonical; expected 0.
completion_after_recovery	root completes after recoverable failure.

8 Experiment 3: Protocol Ablation Study

8.1 实验目的

验证关键机制不是装饰。当前 factorization 最新设计中，最重要的机制不只是 verifier 和 requeue，还包括：插件拥有 prompt / parser、raw-only forbidden、all-required merge gate、slot integrity 和 first-slice limitation enforcement。

8.2 Ablation 矩阵

Mode	Injected condition	Expected degradation
FULL	invalid found factor	verifier rejects; no canonical pollution; root completes after recovery.
NO_VERIFICATION	invalid found factor	wrong range result may become canonical; merge may fail later or wrong canonical risk increases.

NO_PARSER_POLICY	free-form raw output	raw-only / free-form claim may become candidate output; parse boundary broken.
NO_QUEUE	worker crash	expired unit unresolved; root stuck; no settlement.
NO_ALL_REQUIRED_MERGE_GATE	factor slot canonical	merge starts before all required ranges canonical; violates first-slice limitation.
NO_SLOT_INTEGRITY_CHECK	slot mismatch	range result can be bound to wrong required slot; merge trust boundary broken.

8.3 特别注意：不再把 early success 当作已实现能力

当前第一切片明确不实现 `one_success`、early terminal resolution 或 sibling pruning。因此消融实验不能写成“TokenShare 应该找到一个 factor 后立即停止”。正确写法是：

FULL mode 必须证明 semiprime factor-found partial run 在所有 required ranges canonical 前不会创建 merge task、不会写 merge record、不会 settlement。NO_ALL_REQUIRED_MERGE_GATE 才故意破坏这个约束，用来展示 all-required merge gate 的必要性。

8.4 Experiment 3 指标

Metric	Definition
wrong_canonical_acceptance_rate	invalid outputs bound as canonical / injected invalid outputs.
raw_only_acceptance_rate	raw-only or free-form outputs treated as successful candidate outputs / raw-only cases.
stuck_task_rate	runs with unresolved required units / total runs.
premature_merge_rate	merge started before all required slots canonical / partial runs.
slot_mismatch_acceptance_rate	wrong slot binding accepted / slot mismatch injections.
completion_rate	completed runs / total runs.

9 Experiment 4: Cross-Plugin Generality with Real Plugins

9.1 实验目的

Experiment 4 继续测试协议泛化能力，但必须使用真实插件。也就是说：

- factorization 侧使用 `factorization@0.1.0` 真实插件。

- Lean 侧使用真实 Lean proof plugin / Lean adapter，不是 lean_stub。
- 如果 Lean plugin 尚未实现或本地没有可复现 Lean / lake 环境，则 Experiment 4 不能声称通过，只能标记为 blocked 或 pending real Lean plugin。

9.2 两个真实插件的对比维度

Dimension	factorization plugin	Lean proof plugin
task domain	integer factorization / bounded arithmetic search	formal theorem proving
root input	RootInput(target_n)	Lean theorem / proof-state code artifact
split authority	plugin deterministic candidate_range_partitioning	plugin deterministic parser / minimizer / split strategy
executor role	bounded range search only	proof candidate generation only
verifier	deterministic factorization verifier	fixed local Lean / lake checker
merge	all-required range merge	verified subproof merge / theorem artifact merge
final output	PrimeFactorizationResult	root proof artifact accepted by Lean
core knowledge	none about divisibility	none about Lean tactic / theorem semantics

9.3 Shared lifecycle stages

两个插件都必须使用同一套协议对象和事件：

```

PluginDescriptor
RegistrySnapshot
TaskSpec / TaskUnit
ArtifactRef / ArtifactStore
ExecutionRequest
ExecutionSubmission
VerificationReport
CanonicalOutputBundle
SplitStrategyInvocation
DecompositionProposal
ExpansionDecision
MergePlan
MergeTaskLink
MergeRecord
ExpectedOutputResolution
ContributionRecord

```

```
SettlementRecord
EventLedger replay
```

9.4 Lean 插件最低通过条件

Lean 插件不能只返回 success。最低必须满足：

1. 输入是 Lean theorem / proof-state code artifact，而不是自然语言命题。
2. 插件内 deterministic parser / normalizer / split strategy 识别 Lean 命题结构并生成 proposal / merge plan。
3. AI 或 executor 不能决定协议级子任务，只能提交 proof candidate。
4. verifier 使用固定本地 Lean toolchain / lake project / import context 检查 proof artifact。
5. **EnvironmentRef** 记录 Lean executable、toolchain、lake project、library/import set、namespace/context、config digest 和摘要。
6. checker stdout/stderr、exit code、timeout、diagnostics、normalized theorem/proof digest 和 proof artifact ref 都必须持久化。
7. replay 不重新运行 Lean 来补历史事实，只根据 event / artifact 重建状态。

9.5 Experiment 4 执行流程

Step	操作	检查
1	运行 factorization fixture	使用 Experiment 1 的 semiprime fixture，完成 full lifecycle。
2	运行 Lean direct proof fixture	真实 Lean checker 验证一个 direct proof artifact。
3	运行 Lean decomposition / merge fixture	Lean plugin 自动拆分，子证明 artifact 通过 checker，merge 后 root theorem 再次通过 checker。
4	比较 shared event coverage	两个插件均覆盖 shared lifecycle stages。
5	检查 core boundary	core 中无 factorization divisibility 逻辑，无 Lean tactic / checker / proof merge 逻辑。
6	descriptor replay	两个插件 descriptor digest 可 replay 验证。
7	artifact replay	不重新运行 executor / Lean / AI API，仅从 artifacts 和 event log 重建状态。

9.6 Experiment 4 指标

Metric	Definition	Target
--------	------------	--------

shared_protocol_event_coverage	observed shared lifecycle stages / expected stages	1.0
plugin_boundary_purity	no domain-specific logic in protocol core	1
descriptor_replay_success	descriptor digest verifies during replay	1
artifact_replay_success	referenced artifacts load and link correctly	1
lifecycle_equivalence_score	common lifecycle stages completed by both plugins / expected stages	1.0
real_checker_evidence	Lean side has real checker logs and environment ref	1

10 推荐 CLI / 测试入口

在实验基础设施尚未完全落地前，优先使用仓库已有测试作为 ground truth:

```
# Factorization plugin unit tests
PYTHONPATH=src python -m pytest tests/plugins/factorization -q

# Factorization real plugin end-to-end fixtures
PYTHONPATH=src python -m pytest tests/test_phase6_factorization_flow.py -q

# First-slice limitation tests
PYTHONPATH=src python -m pytest tests/test_phase6_factorization_limitations.py -q

# Phase 7 AI API executor tests
PYTHONPATH=src python -m pytest tests/executors tests/test_phase7_ai_api_execution_flow.py -q

# Full startup validation
./init.sh
# or on Windows:
.\init.ps1
```

未来 ExperimentRunner CLI 可设计为:

```
python -m tokenshare.experiments.exp1_factorization_real_plugin \
  --fixture semiprime --executor deterministic_local --seed 1

python -m tokenshare.experiments.exp1_factorization_real_plugin \
  --fixture semiprime --executor ai_api --seed 1 --allow-smoke-skip

python -m tokenshare.experiments.exp2_factorization_failures \
  --case false_no_factor --seed 1

python -m tokenshare.experiments.exp3_factorization_ablation \
  --mode NO_ALL_REQUIRED_MERGE_GATE --seed 1
```

```
python -m tokenshare.experiments.exp4_real_plugin_generality \
  --plugins factorization,lean_proof --seed 1
```

11 最小验收标准

更新后的实验设计通过，当且仅当：

1. factorization descriptor 使用 `plugin_version=0.1.0`，并声明最新 strategy / parser / validator / merge policy。
2. root canonical output bundle key 是 `factor_integer_subject`，不能把 root subject 伪装成最终 `prime_factorization_result`。
3. split strategy 生成完整候选域 coverage proof，且 range children 无 gap / overlap。
4. range child request 同时持久化 `execution_instruction_ref` 和插件拥有的 `prompt_package_ref`。
5. AI / mock-AI raw output 必须经过插件 parser；free-form / raw-only 不能进入 verification 或 canonical。
6. invalid found factor、false no-factor、coverage mismatch、slot mismatch 和 budget exceeded 都被 deterministic verifier / merge policy 拒绝。
7. semiprime fixture 必须等所有 required ranges canonical 后才能 merge；不得宣称 early success 或 sibling pruning。
8. composite cofactor case 只能输出 `nontrivial_factor_found` 和 limitation reason，不能解析成 final result。
9. Phase 7 AI API executor 只负责 provider transport、raw artifact、provenance、usage/cost 和插件 parser bridge；不拥有 factorization 解析权。
10. Experiment 4 必须使用真实 Lean plugin / Lean adapter。没有真实 checker logs 和 `EnvironmentRef` 时，不得声称跨插件泛化实验已通过。

12 给 AI / Codex 的实现提醒

不要再实现一个脱离仓库对象的 toy range demo。实现时必须复用现有 `src/tokenshare/plugins/factorization/ProtocolEngine`、`ArtifactStore`、`EventLedger`、Phase 4 verification / canonical / expansion flow 和 Phase 5 merge / settlement flow。

如果要生成论文实验表格，优先从 JSONL event log 和 artifact refs 复算，而不是在实验脚本里直接硬写成功指标。每一个“通过”都应能追溯到：descriptor digest、artifact digest、verification report、canonical binding、merge record 和 settlement record。